

Week 39 exercises

Marie Océane Pilskog, Muhammad Mohsin Mumtaz, and Synne Sigstad Jørgensen
(Dated: October 5, 2026)

The aim of Project 1 was to investigate a set of regression methods in great detail. Among these the regression of the following methods have been studied: Ordinary Least Squares (OLS), Ridge, and Lasso. Studies have been performed of bias-variance tradeoff including its derivation, as well as bootstrap and cross-validation. The starting point for the analysis has been the Runge function. The test error was found to minimize at the following degree []. It was shown that [] is the most effective method for reproducing the closed-form solution. When investigating the Ridge function, an optimal value for the penalty parameter was found at $[\lambda =]$.

I. INTRODUCTION

Machine learning (ML) is a fast growing field. Today, impacts of the advancements in machine learning are seen in numerous domains such as medicine, humanities, and technology. The principle of Machine Learning is to give us the possibility to find patterns in sets of data without explicitly writing out a specific code for each problem we would like to solve [1]. The idea is that our generic algorithm can “learn” from the data we give it.

A. Runges function

Runge’s function, given in Equation 1 as a classic test case when working with ML. The reason for choosing this function as our function to investigate is a concept we call *Runge’s phenomenon*. Discovered by Carl Runge in 1901 [2], the phenomenon describes how when the function is fitted to higher polynomial degrees, we see oscillation at the edges. Using Runge’s function is therefore an excellent example to use when one wants to demonstrate that fitting to a higher polynomial degree is not necessarily synonym with improved results. The reason for this is not that polynomial interpolation is an unfit method, but rather a consequence arising from using a high-degree polynomial built from equally spaced points [3]. This is something we will go into much further detail in this paper.

In this paper, we take a deep dive into Runge’s function, and its characteristics. We wish to investigate how our models behave when we fit the function with regression, especially focusing on how fitting to higher polynomial degrees affect our results.

B. Methods of Regression

One of the desired outputs when working with ML is *regression*: a process where we fit a model to a data set. There are many different approaches to performing regression, some of them we will look at in this paper.

Ordinary Least Squares (OLS) is a practical and a fairly straight forward method of regression, where the principle is that you fit your data by minimising the sum

of the squared errors [should have citation here](#). We can solve for the coefficients of a model using Equation 2.

Ridge regression follows many of the same principles as the regression with OLS, but we add what we call a *penalty parameter*, to the squared length of the parameter vector [should I have reference here?](#). The coefficients can be solved for using Equation 3.

Lasso regression is different from the Ridge Regression model; this one relies on a 1-norm penalty instead of a 2-norm penalty. The model coefficients do not have a closed form solution, but are defined and found by optimising their cost function, defined in Equation 4. Deriving the cost function for Lasso Regression and optimising it gives us Equation 5.

C. Resampling methods

If we have a function $\hat{\theta} = \hat{\theta}(\mathbf{X})$, where $\mathbf{X}$ is random, this means that $\hat{\theta}$ is itself a random variable with an associated PDF $p(x)$ [1]. This means that we can draw a certain number of data points from the function, learn our desired characteristics from it, and interpret the results. This can be especially useful if our sample size is small. There are various methods of resampling. The *Bootstrap* method is a method where you draw random samples from your data, with replacement of the drawn observations, from which you can infer statistical quantities. The *Jackknife* method is a method where you purposely leave out one observation at the time, and then drawn new samples. Both methods require independent, identically distributed variables [1].

A third method for resampling is the *k-fold cross validation* method. Here, you organise the splitting between the training data and the test data, to make sure that no samples land in one of the two categories more than any other sample [1]. You divide the data into so-called *k – folds*, and then the fold serves as the test data, while the remaining data points serve as the training data.

D. Optimisers

Optimisers are algorithms that change the weights on our model. In this paper, we have used the optimisers

momentum, *adagrad*, *RMSprop*, and *adam*. These methods are ways of optimising the cost function of our model using different parameters to retrieve better results for our model coefficients.

This paper is structured as follows: In [section II](#) we will present the theoretical background used for our work, as well as relevant equations. In [section III](#) we will display the methods we have applied to achieve our results, which will be presented and discussed in [section IV](#). Our conclusions can be found in [section V](#).

The code developed for this paper can be found on our GitHub repository ¹.

II. THEORY

Note: the theory section will be written more coherent, for now it is just so that the references in the introduction work!

A. Important equations for regression

Runge's function is given by

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1]. \quad (1)$$

When using OLS, we can solve for the model coefficients $\hat{\theta}_{OLS}$ via the equation

$$\hat{\theta}_{OLS} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}, \quad (2)$$

where $\mathbf{X}$ is the design matrix given by $\mathbf{X} \in \mathbb{R}^{n \times p}$ for n observations and p features, $\mathbf{y} \in \mathbb{R}^n$ is the function outputs from our discrete data points, and the superscript T means the transpose.

For Ridge regression, we use OLS as our starting point, and then add a penalty parameter, which results in an equation for the parameters given by

$$\hat{\theta}_{Ridge} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}, \quad (3)$$

where $\mathbf{I}$ is the identity matrix, λ is the penalty parameter, and parameters repeating from [Equation 2](#) have the same meaning.

The cost function for Lasso regression is given by

$$C(\theta) = \frac{1}{n} \|\mathbf{y} - \mathbf{X}\theta\|_2^2 + \lambda \|\theta\|_1, \quad (4)$$

where n is the amount of data points and λ is yet another penalty parameter. Differentiating this function gives us the expression

$$2\mathbf{X}^T \mathbf{X}\theta + \lambda \text{sgn}(\theta) = 2\mathbf{X}^T \mathbf{y}, \quad (5)$$

where $\text{sgn}(\theta)$ is defined as a vector with all the signs of the coefficient vector θ .

B. The bias-variance tradeoff

1. Bias, variance, and irreducible error

Bias is the difference between the expectation value for the predicted value by our model, and the true value, given by

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta, \quad (6)$$

where the hat denotes the predicted value, and the **plain** θ is the true value. In other words, the bias is a measure on how simplified our model is [\[4\]](#). If the model is oversimplified, it will miss patterns, and ends up under-fitting. This happens when we have high bias. When we have lower bias, the model is closer to the true values, and it captures patterns well. *Variance* is an error arising from the model being too sensitive to the training data, which might lead to overfitting as discussed [somewhere in the text, make sure this is correct](#). The variance is given by

$$\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]. \quad (7)$$

The bias and the variance combine to give us the mean squared error, given by

$$\text{MSE} = \mathbb{E}[(\hat{\theta} - \theta)^2]. \quad (8)$$

The irreducible error σ^2 is a variance of the noise. Because the noise is random, will σ^2 be a floor beneath the test error. This means that the total test error is a combination of the bias, the variance, and this irreducible error:

$$\text{test error} = \text{bias} + \text{variance} + \sigma^2. \quad (9)$$

The *bias-variance tradeoff* describes the balance between the bias and the variance. A simplified model leads to a bias increase, but a variance decrease, because the model is not too sensitive to the training data. On the other hand, if we make the model complex the bias is lowered, but the variance error is larger, because we have now fit very closely to the training data. The goal is to find a middleground between these errors.

2. The bias-variance trade-off derivation

Assume you have a noisy model $\mathbf{y} = f(\mathbf{x}) + \epsilon$, where $\epsilon \sim N(0, \sigma^2)$. Further assume that you have a data set $\tilde{\mathbf{y}}$ predicts y , we try to find the expected value of the mean square error $E[(\mathbf{y} - \tilde{\mathbf{y}})^2]$. **not done yet...**

¹ link to github

C. Scaling and intercept

Sometimes, the data we use in our model have very different orders of magnitude, or different units. For example, if we have a data set of distances, where some measurements are in the units of meters and some are in the units of kilometers, the measurements will be penalised differently. Centering and scaling of the data effectively make the input features comparable! [used chat gpt here for explanations](#). When they are, the penalty can treat them more fairly. It is common to scale the design matrix so that every column has zero mean and unit standard deviation, using

$$x_j^{(i)} \rightarrow \frac{x_j^{(i)} - \bar{x}_j}{\sigma(x_j)}, \quad (10)$$

where x is a measurement, i denotes the row, j denotes the column, the overline denotes the mean, and σ is the standard deviation. In addition to scaling the design matrix, we should also center the y -values. When we do this, we essentially remove the intercept, so we no longer have a baseline level for our model. The scaling of the y -values can simply be done as

$$y_j = y_j - \bar{y}_j. \quad (11)$$

III. METHODS

IV. RESULTS AND DISCUSSION

A. Ordinary least squares: mean square error, coefficient of determination, and polynomial degree

Figure 1 shows us the MSE values and R^2 (the dimensionless *coefficient of determination*) for our chosen range of polynomial degrees. For the training data, we find from the MSE values that the optimal polynomial degree is 14, while for the test data we find a polynomial degree 7. This difference is expected, as for the training MSE values we train our model on a certain set of noise values, and the higher the polynomial the better the fit for that specific noise data set. However, when we then use the trained model to fit for the test data with new noise data points, the model is fitted so closely with the noise data points for the training data, that the model may no longer be a good fit for the test data. This is what we call *overfitting*, and shows us that the highest polynomial degree is not necessarily the best one.

R^2 compares the model against the best constant prediction [1]. This value of this coefficient lies between 0 and 1. The closer to 1 the better the fit, and vice versa. From Figure 1 we deduce that the values for R^2 agree with the values for the MSE, and gives us the same optimal polynomial degree for both training and test data,

with 14 and 7 respectively. The negative value for the first polynomial degree on the test data, means that the model does even worse than the mean for this degree.

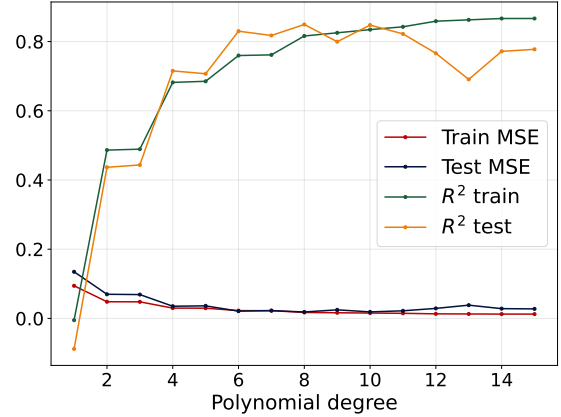


FIG. 1: Caption

Figure 2 displays how the value of θ varies with the polynomial degree, for our test data. For the lower degrees, we see that the values does not vary much as we increase the number of polynomial degrees in our model. However, around a degree of 7 and onward, we see that the values start to vary to a greater extent. This agrees with our earlier findings, that the optimal polynomial degree for our model trained on our given data is 7.

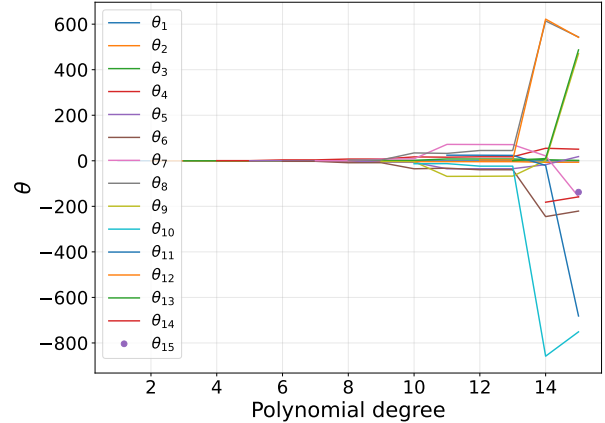


FIG. 2: Caption

B. Ridge Regression

In Figure 3 we have displayed how the values for θ up to a polynomial degree of 5 varies with the penalty parameter λ . As expected, the lowest value for λ should reproduce values for θ close to the values found with the

OLS regression (since we are moving closer to $\lambda = 0$). From our results, it looks like θ_2 and θ_4 is first affected by λ , followed by θ_3 and θ_5 . θ_1 seems to be the coefficient least affected by th change in λ .

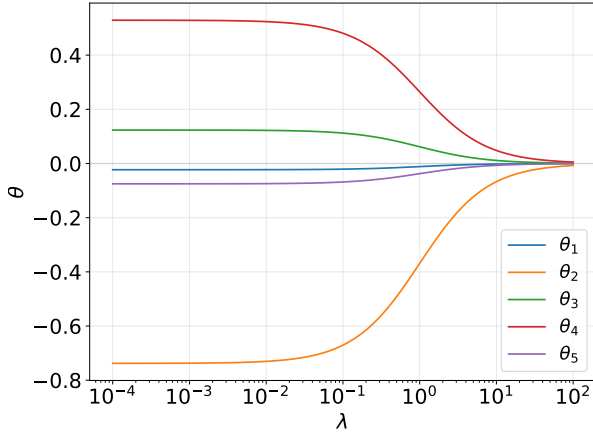


FIG. 3: Caption

C. The bias-variance tradeoff and the bootstrap

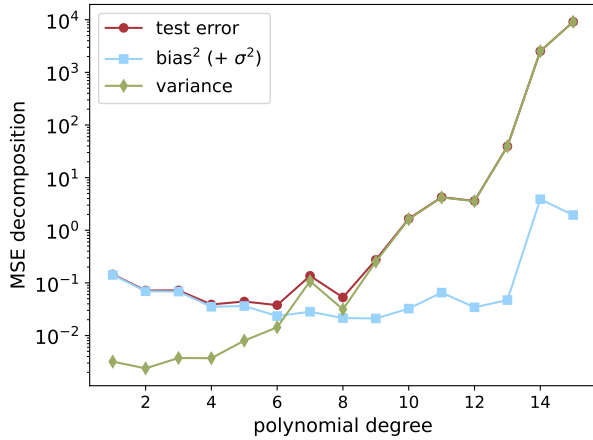


FIG. 4: Caption

D. Cross validation

V. CONCLUSIONS

the conclusion is still very general, as we havent finished the exercises for the project. We will aim to present much more direct results once the exercises have

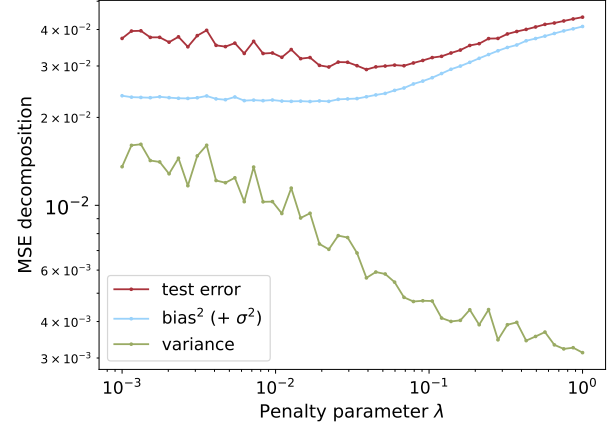


FIG. 5: Caption

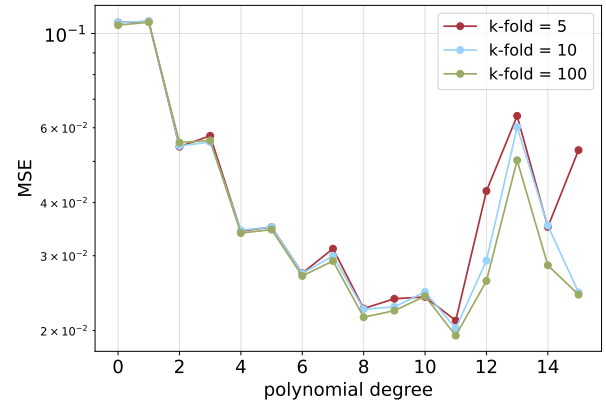


FIG. 6: Caption

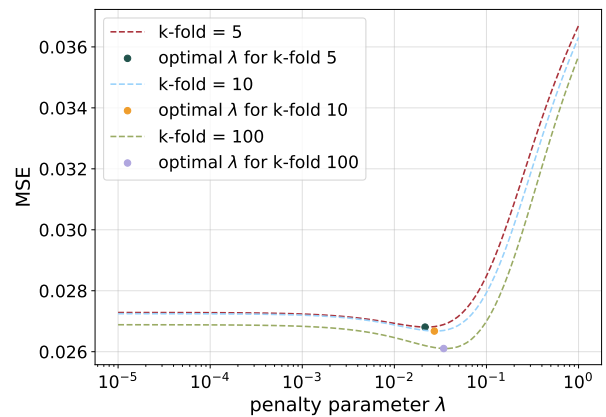


FIG. 7: Caption

been solved.

We have in this paper performed a comprehensive and

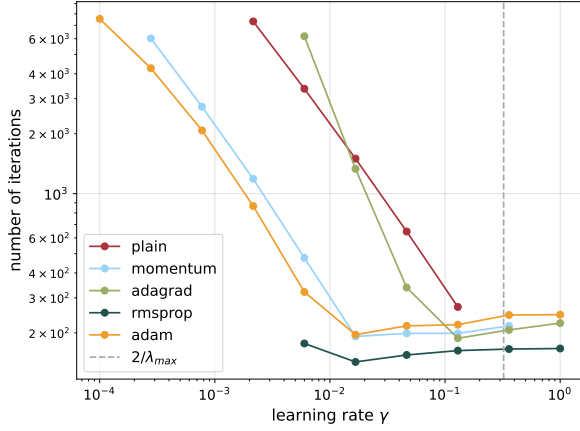


FIG. 8: Caption

thorough analysis of regression, resampling, and optimisation. With the Runge function as a classical test function and starting point, we have visited OLS, Ridge, and Lasso regression, and looked at the mean square error, and the penalty parameter λ . We found that for our function, the optimal polynomial degree is 2. We have visited multiple resampling methods, namely the *boot-*

strap method and the *k-fold cross validation* method. We have also investigated various optimisation algorithms, and found that the so-called *adam* algorithm was the least sensitive to λ .

Topics for further work include ...

VI. FIGURES

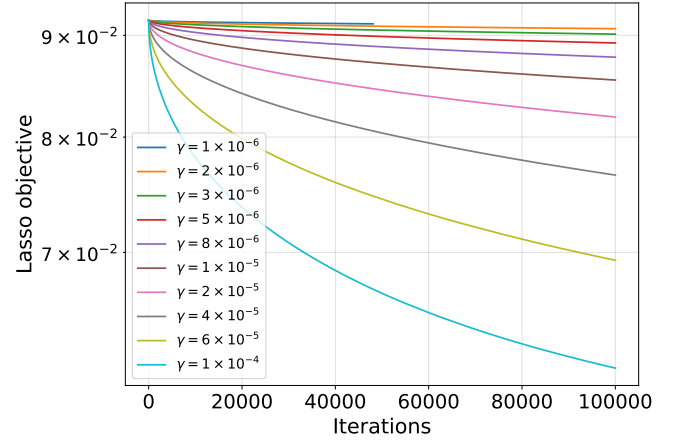


FIG. 9: Caption

-
- [1] Morten Hjort-Jensen. Lecturebook. <https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/BookML/Lecturebook.pdf>. MachineLearningUiO, accessed 22 September 2026.
 - [2] C. Runge. Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten. *Zeit. Math. Phys.*, 46:224–243, 1901.
 - [3] interpolationcal. Runge's Phenomenon: Why High-Degree Polynomials Fail. <https://interpolationcalc.com/blog/runges-phenomenon/>, June 2026. Accessed 23 September 2026.
 - [4] Bias and Variance in Machine Learning. Section: Technical Scripter.

Appendix A: Use of AI/LLM Tools

In our work for project 1, we have written the code ourselves and have not used any AI or LLM Tools for generating code or text.

1. Tools used

No use of AI/LLM tools.

2. Text writing and editing

The report has level 0 of AI/LLM tool use.

3. Code generation and assistance

The developed code has level 0 of AI/LLM tool use.